

Heterogeneous Computational Platform for Parallel Graph Processing

Vladimir M. Kozlov

© The Author(s)

Abstract Efficient graph partitioning is a key factor affecting the performance and scalability of distributed graph databases. The placement of graph vertices across multiple storage nodes determines the amount of inter-shard communication required during graph traversal and shortest-path queries. Modern partitioning algorithms can be roughly divided in two groups: offline and online, in which former take significantly more time than the latter but provide better results. This paper presents a hybrid graph partitioning approach that combines the online partitioning of Fennel algorithm and refinement approach used in METIS framework. The proposed method first constructs an initial balanced partitioning using Fennel and subsequently applies KL-based local optimization to reduce the number of edges crossing shard boundaries while preserving partition balance. This combination leverages the scalability of streaming partitioning and the optimization capabilities of iterative local refinement.

Keywords graph partitioning, distributed graph databases, streaming graph partitioning, Fennel, Kernighan–Lin algorithm, graph algorithms

1 Introduction

Graph-structured data have become a fundamental component of modern information systems, including social networks, recommendation engines, knowledge graphs, transportation networks, and communication infrastructures. As the size of these graphs continues to grow, graph databases increasingly rely on distributed architectures in which graph data are partitioned across multiple storage nodes. While distribution enables horizontal scalability, it also introduces the challenge of efficiently assigning graph vertices to partitions.

The quality of graph partitioning has a direct impact on the performance of distributed graph databases. During graph traversals and shortest-path queries, communication between storage nodes is required whenever adjacent vertices reside in different partitions. Such inter-partition communication

incurs significantly higher latency than local memory access and can become the dominant factor limiting system performance. Consequently, an effective partitioning strategy should simultaneously minimize the graph cut while maintaining a balanced distribution of data across storage nodes.

Numerous graph partitioning techniques have been proposed to address this problem. Some graph databases, such as ArangoDB [?], prioritize balanced data distribution using hash-based partitioning rather than optimizing partition quality. Traditional partitioning frameworks, such as METIS [?], produce high-quality partitions by exploiting the structural properties of the graph. However, these methods are primarily designed for static graphs and often require global repartitioning, making them computationally expensive for large-scale or dynamically evolving datasets. In contrast, streaming partitioning algorithms, such as Fennel [?], provide scalable and efficient vertex placement as the graph is constructed, balancing partition sizes while reducing the expected graph cut. Despite their computational efficiency, streaming methods are inherently greedy and may produce suboptimal partitions that cannot be improved without additional refinement. Furthermore, because they make decisions without knowledge of the complete graph structure, their partition quality may deteriorate when vertices are processed in an unfavorable order.

This work investigates the potential of a hybrid graph partitioning approach that combines the scalability of the Fennel streaming algorithm with the local optimization capabilities of the Kernighan–Lin (KL) algorithm. In the proposed method, Fennel is employed to construct an initial balanced partitioning of the graph, after which KL refinement is applied to reduce the number of inter-partition edges while preserving partition balance. The proposed combination aims to achieve a favorable trade-off between partitioning efficiency and partition quality, making it suitable for large distributed graph databases where both scalability and communication cost are critical considerations.

The hypothesis of this work is that Fennel provides an initial partitioning of sufficient quality such that a small number of KL refinement iterations can achieve higher partition quality

while substantially reducing computational cost compared to the complete METIS partitioning process.

2 Proposed approach

The approaches to graph partitioning can be broadly divided into two categories: offline and online partitioning. Offline methods assume complete knowledge of the graph structure before partitioning, whereas online methods assign each newly arriving vertex to a partition without requiring the entire graph to be known in advance.

The current state-of-the-art online graph partitioning algorithm is Fennel, which employs a heuristic objective function to determine the partition to which each incoming vertex should be assigned. Omitting the details presented in the original work [?], the objective function can be expressed as

$$g(P) = \sum_{i=1}^k \left[e(S_i, S_i) - p \binom{|S_i|}{2} \right],$$

where P denotes a graph partitioning, S_i is a storage i , $e(S_i, S_i)$ is the number of edges whose endpoints are both contained in S_i , and $p = 2\alpha = m \frac{k^{\gamma-1}}{n^\gamma}$, $1 \leq \gamma \leq 2$.

As noted in the original work, evaluating the complete objective function is unnecessary. Instead, it is sufficient to determine the partition whose objective value increases the most after adding the new vertex. This can be expressed as

$$\delta g(v, S_i) = g(S_1, \dots, S_i \cup \{v\}, \dots, S_k) - g(S_1, \dots, S_k), \quad (1)$$

which can be rewritten as

$$\delta g(v, S_i) = [e(S_i \cup \{v\}, S_i \cup \{v\}) - e(S_i, S_i)] - \left[p \binom{|S_i \cup \{v\}|}{2} - p \binom{|S_i|}{2} \right]. \quad (2)$$

Equation 2 can be evaluated independently for each partition, assuming that each storage node is capable of performing the required computation. Since the graph is assumed to be undirected, the quantity

$$e(S_i \cup \{v\}, S_i \cup \{v\}) - e(S_i, S_i)$$

is exactly the number of edges connecting the newly added vertex v to vertices already contained in partition S_i . The second term depends only on the partition size and can therefore be updated whenever a new vertex is assigned to the partition.

Consequently, the Fennel algorithm produces graph partitions with relatively small edge cuts while requiring only modest computational resources.

Offline partitioning methods generally produce higher-quality partitions but require significantly more computation time and resources. The state-of-the-art framework for offline graph partitioning is METIS, which consists of three stages:

- (1) coarsening;
- (2) initial partitioning;
- (3) uncoarsening and refinement.

During the uncoarsening stage, the partitioning is refined using the Kernighan–Lin (KL) algorithm. Both the coarsening and uncoarsening stages are computationally expensive, and refinement is performed multiple times during uncoarsening. However, the original METIS publication reports that, given a sufficiently good initial partitioning, the KL algorithm typically requires no more than five iterations to achieve its best refinement [?].

Although the original METIS publication observes that only a small number of KL iterations are sufficient when the initial partitioning is of high quality, it does not provide a criteria for determining is such a partitioning. Since Fennel has been shown to produce high-quality partitions while operating in an online setting, this motivates the proposed approach of replacing the three-stage METIS partitioning process with Fennel, followed by occasional refinement using the KL algorithm.

The remainder of this paper evaluates the effectiveness of this hybrid approach and investigates the number of KL refinement iterations required when the initial partitioning is generated by Fennel.

3 Experiments

// TODO

4 Conclusion

A distinctive feature of the presented Teragraph computing complex is deep hardware support for associative processing. This is expressed in the fact that at all levels of complex software, data storage is implemented in large-scale associative memory, and thanks to the microprocessor core with the discrete mathematics instruction set DISC, the problem of associative-address translation is essentially solved.

For each of the proposed eight structural hierarchy levels, its purpose, composition, and functional features are shown. The concept of resource distribution and isolation for ensuring multi-user access within the cloud approach is briefly presented. Currently, development of the TeraCloud cloud platform and the set of system and application libraries for the complex is being completed.

The Graph Processor Cores is a doubled core with Computing Processing Element and Structure Processing Ele-

ment, demonstrates heterogeneous efficiency. On one hand, it surpasses the general-purpose CPU by 12 times in clock efficiency when processing sequential data. On the other hand, GPC's performance when working with random keys is 2-4 times lower than that of optimized software data structures on the same CPU, even accounting for the frequency difference.

The key problem is a technological limitation: achieving parity across all operations requires a frequency of 1800 MHz, which is unattainable for modern domestic FPGA and ASIC processes. Therefore, the path of simple frequency scaling for the B+ tree architecture is a dead end.

This necessitates a fundamentally new approach. The development of hybrid data structures is required, combining the efficiency of trees for sequential operations with high-speed handling of random keys. The architecture must include optimized cache memory with intelligent prefetching, mechanisms for speculative execution of traversal operations, and pipelined processing to hide memory access latencies. A critical requirement becomes the transition to multi-threaded and multi-context architectures with fast context switching, which will allow for the most efficient utilization of processor resources when working with irregular graph structures.

The Teragraph system represents a novel approach to graph processing architecture, combining specialized hardware for discrete mathematics operations with traditional computing elements in a heterogeneous configuration. Future work will focus on expanding the instruction set, improving memory hierarchy efficiency, and developing advanced programming tools for the platform.

Acknowledgments

// TODO:

Declaration of Competing Interest

// TODO: